07. 스킬 시스템

사람이 새로운 업무를 배우면, 처음에는 시행착오를 겪지만 경험이 쌓이면 점점 빨라집니다. Hermes Agent도 마찬가지입니다. 복잡한 작업을 성공적으로 완료하면, 그 과정을 스킬(Skill)로 저장해 두고 다음에 재사용합니다. 이 장에서는 에이전트의 스킬 시스템을 다룹니다. 스킬은 에이전트가 복잡한 작업의 절차를 문서화하여 재사용하는 메커니즘으로, "성장하는 에이전트"의 핵심입니다.

이 장에서 다루는 내용

- 스킬의 구조와 작동
- 자동 스킬 생성
- 번들 스킬 카탈로그
- Skills Hub 활용
- 실습: 커스텀 스킬 만들기

07-1. 스킬의 구조와 작동

- 스킬이란
- 스킬은 어디에 설치되나
- 3단계 로딩 (Progressive Disclosure)
- SKILL.md 형식
- 스킬이 슬래시 명령어가 된다
- 신뢰 등급
- 보안 스캔

스킬은 `~/.hermes/skills/` 디렉터리에 저장되는 지식 문서입니다. agentskills.io 오픈 표준을 따르며, 사람이 읽을 수 있는 마크다운 형식입니다.

스킬이란

간단히 말해, 스킬은 **"이런 작업을 할 때는 이렇게 해라"**라는 절차서입니다. 에이전트가 복잡한 작업을 성공적으로 완료하면, 그 과정을 스킬로 저장해 둡니다. 다음에 비슷한 작업이 주어지면 이 스킬을 참고하여 더 빠르고 정확하게 처리합니다.

스킬은 어디에 설치되나

스킬은 모두 `~/.hermes/skills/` 아래에 평평하게 놓입니다. 출처는 세 갈래입니다.

- **번들**: Hermes 설치본과 함께 따라오는 스킬입니다. `hermes update` 시 자동 동기화되고, 사용자가 직접 수정한 스킬은 보호됩니다. 자세한 절차는 「업데이트」 절 참고.
- **Skills Hub로 설치**: `hermes skills install <name>`으로 Hub에서 내려받습니다. 다운로드 직후 보안 스캔(`tools/skills_guard.py`)을 거치며, 통과하면 `~/.hermes/skills/<name>/` 디렉터리로 자리잡습니다.
- **에이전트 자동 생성**: 사용자가 같은 작업을 반복하면 에이전트가 백그라운드로 스스로 SKILL.md를 만들어 같은 위치에 넣습니다(7장 「자동 생성」 참고).

각 스킬 디렉터리에는 최소한 `SKILL.md` 파일이 들어 있고, 필요에 따라 `scripts/` · `templates/` · `references/` · `assets/` 같은 하위 폴더가 함께 들어 있습니다.

3단계 로딩 (Progressive Disclosure)

스킬은 토큰을 절약하기 위해 **단계적 공개(progressive disclosure)** 방식으로 로딩됩니다.

- **Level 0 — 시스템 프롬프트의 스킬 인덱스**: 에이전트가 시작될 때 `skills_index`가 시스템 프롬프트에 포함됩니다. 각 스킬의 이름과 설명만 들어있으며, 스킬 하나당 약 50토큰 정도입니다. 이 단계에서 에이전트는 "어떤 스킬이 있는지"만 파악합니다.
- **Level 1 — `skills_list` 도구 호출**: 에이전트가 특정 분야의 스킬이 필요하다고 판단하면 `skills_list` 도구를 호출합니다. 이름, 설명에 더해 **트리거 조건**과 **사용 예시**가 반환됩니다.
- **Level 2 — `skill_view` 도구 호출**: 실제로 스킬을 사용하기로 결정하면 `skill_view`로 SKILL.md 전체 내용을 로딩합니다.

이 방식 덕분에 수십 개의 스킬이 설치되어 있어도, 현재 필요한 것만 컨텍스트에 올라갑니다. Level 0에서는 스킬이 100개여도 약 5,000토큰이면 충분합니다.

SKILL.md 형식

각 스킬 디렉터리에는 `SKILL.md` 파일이 있으며, YAML 프론트매터와 마크다운 본문으로 구성됩니다.

```
---
name: git-commit-helper
version: 1.0.0
author: NousResearch
description: Git 커밋 메시지 작성 및 커밋 절차 안내
trigger: 사용자가 git commit 관련 요청을 할 때
examples:
  - "커밋 메시지 작성해줘"
  - "변경사항 커밋해줘"
trust: builtin
---

## 사용법

사용자가 커밋을 요청하면 다음 절차를 따릅니다.

1. `git status`로 변경사항 확인
2. 변경 내용을 분석하여 커밋 메시지 초안 작성
3. 사용자에게 확인 요청
4. `git add`와 `git commit` 실행
```

프론트매터의 주요 필드는 다음과 같습니다.

필드	설명
`name`	스킬 식별자 (슬래시 명령어 이름으로도 사용)
`version`	버전
`author`	제작자
`description`	Level 0에서 시스템 프롬프트에 표시되는 설명
`trigger`	Level 1에서 반환되는 트리거 조건
`examples`	Level 1에서 반환되는 사용 예시
`trust`	신뢰 등급 (builtin, official, trusted, community)

스킬 디렉터리에는 추가로 다음 폴더를 포함할 수 있습니다.

- `references/` - 참조 문서
- `templates/` - 템플릿 파일
- `scripts/` - 스크립트
- `assets/` - 이미지 등 에셋

원격 실행 백엔드(Docker·Modal·SSH·Daytona)에서 스킬 파일을 어떻게 동기화하는지는 11장 「컨테이너 격리」에서 다룹니다.

스킬이 슬래시 명령어가 된다

설치된 모든 스킬은 `agent/skill_commands.py`를 통해 자동으로 슬래시 명령어로 등록됩니다. 위 예시의 `git-commit-helper` 스킬은 `/git-commit-helper`로 실행할 수 있습니다.

```
> /skills           # 설치된 스킬 목록 보기
> /git-commit-helper  # 특정 스킬 실행
```

신뢰 등급

스킬의 출처에 따라 신뢰 등급이 다릅니다.

등급	설명
`builtin`	에이전트에 내장된 스킬 (Hermes와 함께 배포)
`official`	NousResearch 공식 스킬
`trusted`	검증된 조직이 배포한 스킬
`community`	누구나 배포할 수 있는 커뮤니티 스킬

보안 스캔

Hub에서 설치하는 스킬은 `tools/skills_guard.py`의 보안 스캐너를 거칩니다. 검사 항목은 다음과 같습니다.

- **데이터 유출(exfiltration)**: 외부로 데이터를 보내는 시도
- **프롬프트 인젝션**: 에이전트의 작동을 조작하려는 패턴
- **공급망 공격**: 의존성을 통한 악성 코드 삽입

로컬 LLM 환경에서의 주의사항

로컬 LLM은 스킬의 3단계 로딩을 항상 정확하게 판단하지 못할 수 있습니다. 특히 Level 0에서 적절한 스킬을 선택하는 능력은 모델 크기에 비례합니다. 7B 모델에서는 스킬이 필요한 상황에서도 호출하지 않거나, 관계없는 스킬을 호출하는 경우가 있습니다. 70B 이상의 모델에서 안정적으로 작동합니다.

07-2. 자동 스킬 생성

> ⚠️ ***부분 검증**: v0.16.0 소스 코드(`agent/background_review.py` , `agent/conversation_loop.py` , `tools/skill_manager_tool.py`)를 기준으로 확인했습니다. 백그라운드 리뷰는 누적 반복 횟수에 도달해야 백그라운드 데몬 스레드에서 조용히 실행되므로, 실제 리뷰 동작을 직접 관찰하는 검증은 하지 않았습니다.*

- 백그라운드 리뷰 시스템
- 자동 생성 과정
- 스킬 관리 기능
- 스킬의 자기 개선
- v0.12에서의 백그라운드 리뷰 강화
- 수동 스킬 생성

Hermes Agent의 가장 흥미로운 기능 중 하나는 **에이전트가 스스로 스킬을 생성**하는 것입니다.

백그라운드 리뷰 시스템

자동 스킬 생성은 "5개 이상의 도구 호출 시 자동 트리거"되는 것이 **아닙니다**. 실제로는 **백그라운드 리뷰(background review)** 시스템이 작동합니다.

초기 버전에서는 사용자 메시지에 "스킬을 만들어 보세요"라는 힌트를 직접 주입하는 인라인 넛지 방식이었습니다. 하지만 이 방식은 에이전트가 사용자의 원래 요청보다 넛지에 먼저 반응하는 문제가 있었고, 현재는 **백그라운드 리뷰** 방식으로 바뀌었습니다.

작동 방식은 다음과 같습니다.

1. 에이전트가 대화 중 도구를 반복적으로 호출합니다.
2. 에이전트 루프 반복이 일정 횟수(기본 10회) 이상 누적되면, **리뷰 플래그**가 켜집니다.
3. 에이전트가 사용자에게 **응답을 완료한 후**, 별도의 백그라운드 스레드에서 리뷰 에이전트가 실행됩니다.
4. 리뷰 에이전트는 대화 내용을 검토하고, 재사용 가능한 워크플로가 있으면 `skill_manage` 도구로 스킬을 생성합니다.

핵심은 메인 대화에 **전혀 간섭하지 않는다**는 것입니다. 리뷰 에이전트는 데몬 스레드에서 실행되고, 최대 16회 반복으로 제한되며, 메인 대화 히스토리를 수정하지 않습니다. CLI에서는 터미널에 간단한 요약이 출력되고, 메시징 게이트웨이(Telegram, Discord 등)에서는 "💾 Self-improvement review: Memory updated · User profile updated"와 같은 알림이 채팅으로 전달됩니다.

메모리 리뷰도 같은 메커니즘을 공유합니다. 일정 턴이 지나면 메모리 리뷰가, 도구 호출이 충분히 누적되면 스킬 리뷰가 각각 백그라운드에서 실행됩니다.

에이전트가 스킬 생성이 적절하다고 판단하는 대표적인 상황은 다음과 같습니다.

- **반복 가능한 워크플로**: 같은 패턴의 도구 호출이 여러 번 나타날 때
- **에러 극복 후**: 시행착오를 거쳐 해결 방법을 발견했을 때
- **사용자의 교정 후**: 사용자가 작업 방식을 수정해주었을 때
- **비자명한 절차 발견 시**: 단순하지 않은 작업 절차를 파악했을 때

자동 생성 과정

예를 들어, 에이전트에게 "Docker 컨테이너의 로그를 분석해서 에러 리포트를 만들어줘"라는 작업을 시켰다고 합시다.

에이전트는 다음과 같은 과정을 거칩니다.

1. `docker logs` 명령으로 로그 수집
2. 에러 패턴 분석
3. 리포트 형식으로 정리
4. 파일로 저장
5. (응답 완료 — 사용자에게 결과물 전달)
6. (백그라운드 리뷰 에이전트가 대화를 검토 → "이 워크플로는 재사용 가능하다" → `skill_manage` 도구로 스킬 생성)

다음에 비슷한 요청이 들어오면 이 스킬을 참고하여 더 빠르게 처리합니다.

스킬 관리 기능

에이전트는 `skill_manage` 도구를 통해 스킬에 대해 다양한 기능을 수행합니다.

기능	설명
`create`	새 스킬 생성 (SKILL.md 작성)
`patch`	기존 스킬의 **특정 섹션만** 수정
`edit`	스킬 **전체를 재작성**
`delete`	스킬 삭제
`write_file`	스킬 디렉터리에 파일 추가 (references, templates 등)
`remove_file`	스킬 디렉터리에서 파일 제거

`patch`와 `edit`의 차이가 중요합니다. `patch`는 스킬의 일부분(예: 에러 처리 섹션)만 업데이트하고, `edit`는 스킬 전체를 새로 씁니다.

스킬의 자기 개선

이미 존재하는 스킬을 사용하다가 더 나은 방법을 발견하면, 에이전트가 `patch`나 `edit` 기능으로 스킬을 개선합니다. 이것이 "자기 개선(self-improving)" 메커니즘의 핵심입니다.

예를 들어:

1. 1회차: Docker 로그 분석 스킬 생성 (`create`)
2. 2회차: 사용 중 에러 처리 로직 추가 (`patch` — 해당 섹션만 수정)
3. 3회차: 출력 형식을 완전히 변경 (`edit` — 전체 재작성)

시간이 지날수록 스킬의 품질이 향상됩니다.

v0.12에서의 백그라운드 리뷰 강화

v0.12에서 백그라운드 리뷰 포크가 한 번에 크게 정비됐습니다. 눈에 보이는 변화는 아니지만, 자동 스킬 생성·개선의 품질에 영향을 주므로 정리해 둡니다.

- **클래스 우선 루브릭** — "이 스킬을 갱신해야 할까?"라는 자유 형식 판단 대신, 기준에 따라 등급을 매기는 루브릭 기반 평가로 바뀌었습니다. 결과가 더 일관됩니다.
- **방금 로드한 스킬을 우선 갱신** — 같은 대화에서 직전에 로드된 스킬이 있으면 새 스킬을 만들기보다 그 스킬을 다듬는 쪽을 선호합니다. 라이브러리가 사용 중심으로 진화합니다.
- `references/`·`templates/` **하위 파일 처리** — SKILL.md 한 장만이 아니라 함께 묶여 있는 참조·템플릿 파일도 갱신 대상이 됩니다.

- **부모 런타임 상속** — 리뷰 포크가 메인 대화의 프로바이더·모델·자격증명을 그대로 물려받습니다(이전에는 일부 환경에서 다른 모델로 폴백되어 품질이 흔들렸습니다).
- **도구 셋 한정** — 리뷰 포크는 `memory`와 `skills` 도구 세트만 활성화됩니다. 셸·웹·터미널로 새는 일이 없습니다.
- **이전 턴 도구 결과 제외(사용자 알림)** — 리뷰 결과로 사용자에게 보여 주는 동작 요약에서는, 메인 대화에 이미 있던 도구 결과가 빠집니다. 직전 대화의 "생성됨·갱신됨" 메시지를 새 동작인 양 다시 띄우지 않습니다.

이 변경의 영향이 가장 잘 드러나는 곳은 백그라운드 큐레이터입니다. 큐레이터는 같은 리뷰 포크 메커니즘을 라이브러리 전체에 주기적으로 돌리는 역할을 하며, 7-6절에서 별도로 다룹니다.

로컬 LLM 환경에서의 자동 생성

로컬 LLM에서는 백그라운드 리뷰 에이전트도 같은 로컬 모델을 사용하므로, 스킬 생성 판단의 품질이 모델 크기에 따라 크게 달라집니다. 불필요한 스킬이 생기거나, 반대로 유용한 기회를 놓칠 수 있습니다. 자동 생성된 스킬은 `~/.hermes/skills/`에서 직접 확인하고, 품질이 낮으면 수동으로 편집하거나 삭제하는 것이 좋습니다.

수동 스킬 생성

자동 생성 외에 직접 스킬을 만들 수도 있습니다.

```
> 새로운 스킬을 만들어줘.
> 이름: daily-report
> 내용: 매일 시스템 상태(디스크, 메모리, CPU)를 점검하고
>       결과를 마크다운 리포트로 작성하는 절차
```

또는 직접 `~/.hermes/skills/daily-report/SKILL.md` 파일을 만들 수도 있습니다.

07-3. 번들 스킬 카탈로그

> ⚠️ ***부분 검증**: 이 페이지의 번들 스킬 목록은 2026년 6월 번들(`~/.hermes/skills/`)을 기준으로 재대조했습니다. 번들 구성은 버전마다 바뀌므로(니치·ML 스킬이 옵션으로 빠지거나 새 스킬이 들어오는 식), 정확한 현재 목록은 환경별로 `hermes skills` 또는 `ls ~/.hermes/skills/` 로 확인하세요.*

- 번들 스킬
  - 개발 및 협업
  - 생산성
  - 추론·평가·데이터
  - 사전 학습 모델
  - 미디어·디자인 — 영상·시각
  - 미디어·디자인 — 디자인·HTML·문서
  - 미디어·디자인 — 음악·오디오
  - 미디어·디자인 — 인터랙티브·창작 워크플로
  - Apple 생태계 (macOS 전용)
  - 리서치
  - 스마트홈
  - 이메일·소셜
  - 레드팀
- 번들 플러그인
- 옵션 스킬 (Skills Hub)
- 변경 이력
  - 번들 정리 (2026년 6월)
  - v0.12.0 — 신규 번들 및 번들 승격

Hermes Agent를 설치하면 번들 스킬이 `~/.hermes/skills/` 디렉터리에 함께 설치됩니다. v0.17.0 기준 번들 스킬은 71개입니다. 플랫폼에 따라 자동 활성화·비활성화되며(예: Apple 스킬은 macOS에서만 활성화), 설정 파일에서 개별 제어할 수도 있습니다. Linux에서는 macOS 전용 스킬이 숨겨져 더 적게 표시됩니다(예: Ubuntu에서는 71개 중 5개가 빠진 66개가 노출).

이 절은 번들 스킬을 카테고리별로 정리하고, 그 뒤에 번들 플러그인·옵션 스킬·변경 이력을 따로 묶습니다. 각 스킬의 상세 사용법은 SKILL.md 파일이나 `/skills` 명령으로 확인할 수 있습니다.

번들은 슬림해지는 방향으로 정리됐습니다

초기에는 파인튜닝·추론 프레임워크나 게임·니치 연동까지 번들에 들어 있었지만, 최근 버전에서 이런 스킬들은 **옵션 스킬**로 빠졌습니다. 기본 설치를 가볍게 유지하고, 필요한 사람만 Skills Hub에서 받아 쓰도록 한 것입니다. 아래 「옵션 스킬」 절과 「변경 이력」을 함께 보면 어떤 스킬이 옵션으로 이동했는지 알 수 있습니다.

번들 스킬

개발 및 협업

소프트웨어 개발, 멀티 에이전트 오케스트레이션, GitHub 연동, DevOps 작업을 한 카테고리로 묶었습니다.

스킬	기능
`plan`	작업 계획 수립·태스크 분해 및 기술 설계 문서 작성
`test-driven-development`	TDD 워크플로 가이드
`systematic-debugging`	4단계 체계적 디버깅
`spike`	일회성 실험 코드(스파이크) 워크플로
`python-debugpy`	debugpy 기반 Python 디버깅
`node-inspect-debugger`	Node 디버거
`requesting-code-review`	PR 리뷰 요청 자동화
`simplify-code`	코드 단순화 — 재사용·중복 제거를 위험도 등급별로 적용
`hermes-agent-skill-authoring`	Hermes 스킬 작성 가이드
`hermes-agent`	독립 Hermes 프로세스 생성 및 조율
`claude-code`	Claude Code에 코딩 작업 위임
`codex`	Codex CLI에 코딩 작업 위임
`opencode`	OpenCode CLI에 코딩 작업 위임
`github-pr-workflow`	PR 생성/커밋/머지/CI 모니터링
`github-code-review`	GitHub PR 코드 리뷰
`github-issues`	이슈 생성/검색/닫기
`github-repo-management`	저장소 관리 및 설정
`github-auth`	GitHub 인증 설정
`codebase-inspection`	코드베이스 분석 (LOC, 언어 분포)
`kanban-orchestrator`	칸반 보드에 작업을 풀어 다른 프로필에 분배

스킬	기능
`kanban-worker`	풀려 있는 작업을 받아 처리
`dogfood`	웹 앱 탐색적 QA — 버그·증거·리포트 생성

생산성

스킬	기능
`google-workspace`	Gmail, Calendar, Drive, Sheets, Docs 연동
`notion`	Notion 페이지·데이터베이스 API 연동
`obsidian`	Obsidian 볼트 노트 관리
`nano-pdf`	PDF 편집
`powerpoint`	PowerPoint 프레젠테이션 자동 생성
`ocr-and-documents`	OCR 및 문서 텍스트 추출
`airtable`	Airtable 베이스·테이블 자동화
`maps`	OSM·위치 기반 작업 (주변 장소 검색 포함)
`teams-meeting-pipeline`	Teams 회의 요약 파이프라인 — 요약·상태 점검·작업 재실행·MS Graph 구독 관리

추론·평가·데이터

스킬	기능
`vllm`	고처리량 서빙 (PagedAttention)
`llama-cpp`	CPU·Apple Silicon 추론
`obliteratus`	모델 안전 필터 제거 (레드팀용)
`weights-and-biases`	실험 추적·모니터링
`lm-evaluation-harness`	LLM 벤치마크 평가
`huggingface-hub`	모델·데이터셋 허브 검색·다운로드
`jupyter-live-kernel`	Jupyter 커널 연동

파인튜닝·양자화·구조화 출력 프레임워크(`axolotl` , `unsloth` , `trl-fine-tuning` , `peft` , `pytorch-fsdp` , `guidance` , `outlines` , `dspy` 등)는 번들에서 빠져 **옵션 스킬**로 제공됩니다.

사전 학습 모델

번들 스킬 형태로 들어오는 사전 학습 모델입니다.

스킬	기능
`audiocraft`	Meta 오디오·음악 생성
`segment-anything`	SAM 비전 분할 모델

이미지·음성·비전 모델(`stable-diffusion` , `whisper` , `clip`)은 옵션 스킬로 이동했습니다.

미디어·디자인 — 영상·시각

스킬	기능
`youtube-content`	YouTube 자막 추출 및 콘텐츠 분석
`manim-video`	Manim 수학·도해 영상
`ascii-video`	ASCII 비디오 생성
`p5js`	p5.js 인터랙티브 스케치
`excalidraw`	손그림 스타일 다이어그램
`architecture-diagram`	아키텍처 다이어그램
`ascii-art`	ASCII 아트 생성
`gif-search`	GIF 검색 (Tenor)
`baoyu-infographic`	인포그래픽 생성 (레이아웃·스타일 다수 조합)

미디어·디자인 — 디자인·HTML·문서

스킬	기능
`claude-design`	Anthropic Claude.ai 스타일 HTML 아티팩트
`design-md`	Google `DESIGN.md` 스펙으로 기능 설계서 작성
`popular-web-designs`	인기 웹 디자인 패턴 템플릿
`sketch`	빠른 HTML 목업
`pretext`	브라우저용 창작 데모(@chenglou/pretext 기반)
`humanizer`	LLM 흔적이 강한 텍스트에서 AI투를 줄여 자연스럽게

미디어·디자인 — 음악·오디오

스킬	기능
`heartmula`	오픈소스 음악 생성
`songsee`	오디오 스펙트로그램 시각화
`songwriting-and-ai-music`	작곡 보조

Spotify는 스킬이 아니라 게이트웨이 **번들 플러그인**으로 제공됩니다(아래 「번들 플러그인」 참고).

미디어·디자인 — 인터랙티브·창작 워크플로

스킬	기능
`comfyui`	ComfyUI 이미지 생성 워크플로 — 공식 CLI + REST + 하드웨어 점검
`touchdesigner-mcp`	TouchDesigner-MCP 모션·시각 인터랙션

Apple 생태계 (macOS 전용)

스킬	기능
`apple-notes`	Apple 메모 생성·검색·편집
`apple-reminders`	미리알림 관리
`imessage`	iMessage 읽기·보내기
`findmy`	Find My로 기기·AirTag 위치 추적
`macos-computer-use`	macOS 데스크톱 백그라운드 제어 — 스크린샷·마우스·키보드를 사용자 커서·포커스를 뺏지 않고 조작

리서치

스킬	기능
`arxiv`	arXiv 논문 검색 (API 키 불필요)
`research-paper-writing`	ML/AI 논문 작성 파이프라인
`blogwatcher`	블로그·RSS 모니터링
`polymarket`	예측 시장 데이터 조회
`llm-wiki`	Karpathy의 LLM Wiki — 상호 연결된 마크다운 지식베이스 구축·질의

스마트홈

스킬	기능
`openhue`	Philips Hue 조명 제어

이메일·소셜

스킬	기능
`himalaya`	터미널 이메일 클라이언트
`xurl`	xurl CLI로 X/Twitter 자동화 — 게시·검색·DM·미디어 (v2 API)
`yuanbao`	Yuanbao(元宝) 그룹 — 멤버 @멘션·정보·멤버 조회

레드팀

스킬	기능
`godmode`	LLM 안전 필터 우회 테스트

번들 플러그인

`skills/`에 들어가지는 않지만, 번들된 플러그인으로 자동 로드되는 도구가 함께 옵니다. 게이트웨이를 시작하면 자동으로 감지돼 도구·페이지·이벤트 훅에서 사용할 수 있고, 대시보드 Plugins 탭에서 켜고 끌 수 있습니다.

플러그인	기능
Spotify	재생·검색·큐·플레이리스트 7개 도구를 게이트웨이에서 직접 노출 (PKCE OAuth)
Google Meet	음성 채널에 봇으로 참여, 받아쓰기·발화·후속 작업까지 한 파이프라인
Langfuse 옵저버빌리티	LLM 호출·도구 실행을 Langfuse에 기록
`hermes-achievements`	전체 세션 히스토리를 스캔해 진행 성과(일종의 도전 과제) 표시

옵션 스킬 (Skills Hub)

번들에 항상 포함되지는 않지만, Skills Hub의 신뢰된 기본 tap(`huggingface/skills` —「Skills Hub」 절 참고)에서 한 번에 설치할 수 있습니다. 위에서 언급한, 번들에서 빠진 파인튜닝·추론·모델 스킬(`dspy` , `axolotl` , `stable-diffusion` , `whisper` 등)과 게임·니치 연동(`pixel-art` , `minecraft-modpack-server` , `pokemon-player` , `mcporter` , `domain-intel` 등)도 모두 이 경로로 받습니다. 아래는 자주 쓰이는 일부입니다.

스킬	영역
`hyperliquid`	Hyperliquid SDK·REST API 기반 perpetual·spot 거래
`stocks`	실시간 시세·기본 재무·과거 데이터
`rest-graphql-debug`	REST + GraphQL 디버그 레시피
`evm`	Ethereum + L2 등 EVM 체인 통합
`darwinian-evolver`	진화 알고리즘 기반 프롬프트·스킬 튜닝
`osint-investigation`	사람·도메인·조직 대상 OSINT 레시피
`pinggy-tunnel`	로컬 서비스를 공용 인터넷에 노출
`watchers`	RSS / HTTP JSON / GitHub 변화 감지(크론 `no_agent` 모드)

스킬은 계속 바뀝니다

번들 스킬 목록은 버전이 올라갈 때마다 더해지기도 하고, 니치 스킬이 옵션으로 빠지기도 합니다. 현재 설치된 스킬의 최신 목록은 `ls ~/.hermes/skills/` 로 확인하세요. Skills Hub에서 커뮤니티 스킬을 추가로 설치할 수도 있습니

다.

변경 이력

번들 정리 (2026년 6월)

기본 설치를 가볍게 유지하기 위해 번들을 슬림화했습니다.

- **옵션으로 이동**: `dspy`, `pixel-art`, `subagent-driven-development`, `minecraft-modpack-server`, `pokemon-player` 등이 번들에서 빠져 옵션 스킬(`optional-skills/`)로 이동. 앞서 파인튜닝·추론 스킬(`axolotl`, `unsloth`, `trl-fine-tuning`, `peft`, `pytorch-fsdp`, `guidance`, `outlines`)과 모델 스킬(`stable-diffusion`, `whisper`, `clip`), 니치 연동(`mcporter`, `modal`, `domain-intel`)도 같은 방식으로 옵션화됨
- **통합**: `writing-plans`는 `plan` 스킬로 병합, `native-mcp`·`webhook-subscriptions`는 `hermes-agent` 스킬의 참조 문서(`references/native-mcp.md`, `references/webhooks.md`)로 흡수
- **제거·개명**: `linear`는 제거(대신 `hermes mcp install linear`), `spotify` 스킬은 게이트웨이 플러그인으로 일원화, `xitter`는 `xurl`로 개명, `debugging-hermes-tui-commands`·`inference-sh`·`find-nearby`(→ `maps`로 흡수) 제거
- **신규 번들**: `baoyu-infographic`, `dogfood`, `llm-wiki`, `macos-computer-use`, `teams-meeting-pipeline`, `yuanbao`

v0.12.0 — 신규 번들 및 번들 승격

스킬·플러그인	변화
`comfyui`	optional → **번들 기본** (하드웨어 점검 후 로컬 설치 게이트)
`touchdesigner-mcp`	**번들 기본**, GLSL/post-FX/오디오/지오메트리 9종 레퍼런스 추가
`humanizer`, `claude-design`, `design-md`, `pretext`, `spike`, `sketch`, `airtable`	번들 신규
`kanban-orchestrator`, `kanban-worker`	번들 신규
Google Meet, Langfuse, hermes-achievements	번들 플러그인 신규

07-4. Skills Hub 활용

> ⚠️ ***부분 검증**: v0.16.0 소스 코드 분석으로 재확인(`hermes bundles` 명령·skill-bundles 모듈 현존). 실제 실행 테스트는 하지 않았습니다.*

- 소스 어댑터
- 스킬 관리 CLI 명령어
- 핀(pin)과 외부 디렉터리
- 슬래시로 즉시 반영: /reload-skills
- 보안: 격리 → 스캔 → 설치
- 플랫폼별 스킬 활성화/비활성화
- 여러 스킬을 한 번에 호출: 스킬 묶음(hermes bundles)
  - YAML 파일
  - 자주 쓰는 명령
  - 작동 방식
  - 언제 쓰면 좋은가

Skills Hub는 커뮤니티가 만든 스킬을 검색하고 설치할 수 있는 마켓플레이스입니다.

소스 어댑터

스킬은 여러 소스 어댑터를 통해 다양한 곳에서 가져올 수 있습니다.

어댑터	설명
`official`	NousResearch 공식 스킬
`skills-sh`	skills.sh 스킬 허브
`clawhub`	OpenClaw 스킬 허브 (호환)
`lobehub`	LobeHub 스킬
`huggingface`	`huggingface.co/skills` 커뮤니티 인덱스. **v0.14부터 신뢰된 기본 tap으로 등록**되어 별도 설정 없이 검색·설치 가능
`nvidia`	`NVIDIA/skills` — NVIDIA 검증 스킬(CUDA-X, AIQ, cuOpt 등). **v2026.5.29부터 신뢰된 기본 tap으로 등록**되어 별도 설정 없이 검색·설치 가능
`browse-sh`	browse.sh — Browserbase가 운영하는 사이트별 브라우저 자동화 카탈로그(200+ 사이트). 식별자 형식 `browse-sh/<hostname>/<task-id>`, 신뢰 등급 `community`
`github`	GitHub 저장소에서 직접 설치
`local_path`	로컬 파일 경로에서 설치
`url`	URL에서 직접 다운로드하여 설치

`local_path`와 `url` 어댑터가 있어서 공식 허브에 등록하지 않은 스킬도 쉽게 설치할 수 있습니다. 팀 내부 스킬을 비공개 GitHub 저장소나 로컬 경로로 공유할 때 유용합니다. `browse-sh`는 Airbnb·Amazon·arXiv 같은 개별 사이트를 어떻게 다루는지가 SKILL.md로 정리된 카탈로그라, 브라우저 자동화 작업에 짝지어 쓰기 좋습니다.

스킬 관리 CLI 명령어

```
hermes skills browse          # 사용 가능한 스킬 목록 탐색
hermes skills search <query>  # 스킬 검색
hermes skills install <name>  # 스킬 설치 (이름)
hermes skills install <url>   # HTTP(S) URL에서 직접 설치
hermes skills inspect <name>  # 설치 전 스킬 내용 미리 보기
hermes skills list            # 설치된 스킬 목록(활성/비활성 표시)
hermes skills check           # 설치된 스킬 업데이트 확인
hermes skills update          # 스킬 업데이트
hermes skills audit           # 보안 감사
```

`inspect` 명령은 스킬을 설치하기 전에 내용을 확인할 수 있어서, 신뢰하지 않는 소스의 스킬을 검토할 때 유용합니다. v0.12부터는 직접 입력한 HTTP(S) URL에서도 같은 보안 절차(격리·스캔)를 거쳐 설치할 수 있습니다 — 사내 게시판이나 비공개 GitHub Pages 같은 위치에 올린 스킬을 공유할 때 편리합니다.

핀(pin)과 외부 디렉터리

기능	설명
핀	스킬을 `pinned` 상태로 표시하면, 자동 백그라운드 큐레이터(7-6절)가 함부로 손대지 못하고 `skill_manage`도 쓰기를 거부합니다. 사람이 손으로 잘 다듬어 둔 스킬을 보호할 때 사용
`external_dirs`	`config.yaml` 의 `skills.external_dirs` 에 디렉터리를 추가하면 그 폴더의 스킬도 정식 스킬로 인식됩니다. v0.12부터는 `skill_manage` 도구가 이 경로의 스킬도 **그 자리에서 직접 편집**합니다(이전에는 복사본만 수정해 동기화가 깨졌습니다)

슬래시로 즉시 반영: `/reload-skills`

`hermes skills install`이나 디렉터리 이동 등으로 스킬 폴더가 바뀌면, 대화 중에 `/reload-skills`로 즉시 재스캔할 수 있습니다(에이전트 재시작 불필요). 도구 셋의 `skills_reload`로도 같은 기능을 사용할 수 있습니다.

보안: 격리 → 스캔 → 설치

Hub에서 스킬을 설치할 때는 3단계 보안 워크플로가 적용됩니다.

1. **격리(Quarantine)**: 다운로드된 스킬이 임시 격리 영역에 배치됩니다.
2. **스캔(Scan)**: `tools/skills_guard.py`가 다음 항목을 검사합니다.

- 데이터 유출(exfiltration) 시도
- 프롬프트 인젝션 패턴
- 공급망 공격 (의존성을 통한 악성 코드)
- 경로 순회 공격 (`../../` 등으로 설치 영역 밖 접근 시도)

3. **설치(Install)**: 스캔을 통과하면 `~/.hermes/skills/` 에 설치됩니다.

스킬의 신뢰 등급(builtin > official > trusted > community)에 따라 실행 권한이 다르게 적용됩니다.

플랫폼별 스킬 활성화/비활성화

설정 파일에서 스킬을 플랫폼별로 활성화하거나 비활성화할 수 있습니다. 예를 들어, macOS 전용 Apple 스킬을 Linux 서버에서는 비활성화하는 식입니다.

번들 스킬만으로 부족하다면, Skills Hub에서 커뮤니티가 만든 스킬을 추가로 설치할 수 있습니다. 번들 스킬의 전체 카탈로그는 이전 절을 참고하세요.

여러 스킬을 한 번에 호출: 스킬 묶음(`hermes bundles`)

설치된 여러 스킬을 묶어 **하나의 슬래시 명령**으로 한꺼번에 로드하는 기능입니다. 같은 작업에 늘 같은 스킬 조합을 쓴다면, 매번 `/skill-a` · `/skill-b` · `/skill-c` 를 따로 호출하지 않고 `/<묶음 이름>` 한 번이면 됩니다(여기서 말하는 묶음은 7-3절에서 다룬 **내장 번들 스킬**과 다른 개념입니다 — 사용자가 직접 만드는 **스킬 묶음**입니다).

빠른 예:

```
hermes bundles create backend-dev \
  --skill github-code-review \
  --skill test-driven-development \
  --skill github-pr-workflow \
  -d "백엔드 기능 작업 — 리뷰·TDD·PR"
```

이후 CLI나 메시징 게이트웨이에서:

```
/backend-dev auth 미들웨어 리팩터링해 줘
```

묶음에 포함된 스킬 셋이 한 사용자 메시지에 통째로 실려 들어가고, 슬래시 뒤의 텍스트는 사용자 지시로 함께 전달됩니다.

YAML 파일

묶음은 `~/.hermes/skill-bundles/<slug>.yaml` 에 저장됩니다.

```
name: backend-dev
description: 백엔드 기능 작업 — 리뷰·TDD·PR
skills:
  - github-code-review
  - test-driven-development
  - github-pr-workflow
instruction: |
  먼저 실패하는 테스트부터 작성한 다음 구현해 주세요.
  PR은 표준 워크플로로 열고 공동 작성자 태그를 함께 답니다.
```

필드	설명
`name`	묶음 이름. 생략하면 파일 이름의 stem이 사용됨. 하이픈 슬러그로 정규화되어 슬래시 명령이 됨(`Backend Dev` → `/backend-dev`)
`description`	`/bundles` 와 `hermes bundles list` 에서 표시되는 짧은 설명
`skills` (필수)	묶을 스킬 목록. `/<skill-name>` 으로 호출할 때 쓰는 식별자 그대로 적음
`instruction`	스킬 본문 앞에 더해지는 지시. "이 묶음을 쓸 때 늘 이렇게 한다"를 코드화할 수 있음

자주 쓰는 명령

```
hermes bundles list                       # 설치된 묶음 모두 보기
hermes bundles show backend-dev           # 한 묶음 상세
hermes bundles create research            # 대화형 생성 (--skill 없이 실행하면 한 줄에 하나씩 입력)
hermes bundles create backend-dev ... --force   # 기존 묶음 덮어쓰기
hermes bundles delete backend-dev
hermes bundles reload                     # 디렉터리 재스캔 후 변경 사항 보고
```

대화 세션 안에서는 `/bundles` 로 설치된 묶음과 각 묶음에 들어 있는 스킬을 한눈에 볼 수 있습니다.

작동 방식

- **묶음이 스킬보다 우선**: 묶음 슬러그와 스킬 슬러그가 같으면 `/슬러그` 는 묶음을 부릅니다. 같은 이름을 의도적으로 골랐다는 표시로 봅니다.

- **누락 스킬은 건너뜀**: 묶음이 가리키는 스킬 중 설치돼 있지 않은 게 있어도 묶음은 실패하지 않고, 가능한 스킬만 로드하고 건너뛴 목록을 에이전트에 전달합니다.
- **모든 표면에서 작동**: CLI·TUI·웹 대시보드 채팅·메시징 게이트웨이(텔레그램·디스코드 등) 어디서 호출해도 같은 방식으로 작동합니다.
- **프롬프트 캐시를 깨지 않음**: 묶음 호출은 시스템 프롬프트를 바꾸지 않고 새 사용자 메시지를 한 번 만들어 보냅니다(`/<skill-name>` 과 같은 방식).

언제 쓰면 좋은가

- `/release-prep` , `/incident-response` 처럼 같은 작업에 늘 같은 스킬 조합이 들어갈 때.
- 슬래시 명령을 여러 번 두드리는 멘털 모델이 부담스러울 때.
- 팀 단위 "작업 프로파일"을 dotfiles 저장소에 YAML로 체크인하고 `~/.hermes/skill-bundles/` 에 심볼릭 링크로 두는 방식으로 공유하고 싶을 때.

묶음 자체는 YAML 별칭이라 스킬을 함께 설치해 주지는 않습니다. 묶음에 적힌 스킬들은 `~/.hermes/skills/` 나 등록된 외부 디렉터리에 이미 있어야 합니다(없으면 호출 시 건너뜀).

07-5. 실습: 커스텀 스킬 만들기

> ⚠️ ***부분 검증**: 이 페이지의 명령·도구·파일 경로는 v0.16.0 소스 코드 (`hermes_cli/skills_hub.py` · `agent/skill_commands.py` · `tools/skill_manager_tool.py`)로 재확인했습니다. 실습 절차를 직접 실행한 테스트는 하지 않았으므로, 출력 화면과 자동 스킬 생성 동작은 환경·모델에 따라 달라질 수 있습니다.*

- 실습 1: 설치된 스킬 확인
- 실습 2: 스킬 검색 및 설치
- 실습 3: 커스텀 스킬 수동 생성
- 실습 4: 자동 스킬 생성 관찰
- 실습 5: 스킬 내용 확인

이 절에서는 스킬을 직접 만들고, 자동 스킬 생성을 관찰합니다.

실습 1: 설치된 스킬 확인

```
> /skills list
```

설치된 스킬 목록이 표시됩니다. 번들 스킬과 에이전트가 자동 생성한 스킬이 함께 보입니다.

스킬은 `agent/skill_commands.py` 를 통해 슬래시 명령어로 등록됩니다. `/skills list` 명령으로 전체 목록을 볼 수 있고(맨몸 `/skills` 는 도움말을 보여줍니다), 각 스킬 이름을 슬래시 명령어로 직접 실행할 수 있습니다.

알려진 이슈: 스킬 출처 표시 오류

`/skills` 목록에서 로컬에서 만든 스킬이 "builtin"으로 잘못 표시되는 경우가 보고되어 있습니다 (GitHub #861). 스킬이 실제로 어디에 있는지는 `~/.hermes/skills/` 디렉터리를 직접 확인하는 것이 정확합니다.

실습 2: 스킬 검색 및 설치

```
hermes skills search "git"
hermes skills install git-helper
```

또는 대화 중에:

```
> git 관련 스킬을 검색하고 유용한 것을 설치해줘
```

설치 전에 `hermes skills inspect <name>`으로 스킬 내용을 미리 확인할 수 있습니다. 특히 community 등급 스킬은 설치 전 내용을 검토하는 습관이 좋습니다.

실습 3: 커스텀 스킬 수동 생성

시스템 점검 스킬을 직접 만들어 봅니다.

```
> 다음 절차를 수행하는 스킬을 만들어줘:
> 이름: system-check
> 1. 디스크 사용량 확인 (df -h)
> 2. 메모리 사용량 확인
> 3. 실행 중인 프로세스 수 확인
> 4. 결과를 마크다운 테이블로 정리
```

에이전트가 `skill_manage` 도구의 `create` 기능으로 `~/.hermes/skills/system-check/SKILL.md`를 생성합니다. 이후 `/system-check`로 실행할 수 있습니다.

실습 4: 자동 스킬 생성 관찰

복잡한 작업을 시켜서 에이전트가 자동으로 스킬을 생성하는 과정을 관찰합니다.

```
> 현재 디렉터리에서 모든 Python 파일을 찾아서,
> 각 파일의 import 문을 분석하고,
> 사용되는 외부 패키지 목록을 추출하고,
> requirements.txt 파일을 생성해줘
```

이 작업은 여러 도구 호출이 필요합니다. 도구 호출이 충분히 누적되면, 응답 완료 후 백그라운드 리뷰 에이전트가 이 워크플로를 스킬로 만들지 검토합니다.

자동 생성이 발생하지 않을 수도 있습니다. 최종 결정은 LLM이 하기 때문입니다. 만약 자동 생성이 되지 않는다면 다음과 같이 직접 요청할 수 있습니다.

```
> 방금 한 작업을 스킬로 만들어줘
```

로컬 LLM에서의 자동 생성

로컬 LLM(특히 7B~13B 모델)에서는 백그라운드 리뷰의 판단 품질이 떨어져서, 불필요한 스킬을 만들거나 반대로 유용한 기회를 놓칠 수 있습니다. 자동 생성에 의존하기보다는 수동으로 스킬을 만들거나, 자동 생성된 스킬을 직접 검토하고 편집하는 것을 권장합니다.

실습 5: 스킬 내용 확인

생성된 스킬의 내용을 직접 확인합니다.

```
cat ~/.hermes/skills/system-check/SKILL.md
ls ~/.hermes/skills/
```

스킬 파일은 일반 마크다운이므로, 텍스트 에디터로 직접 수정할 수도 있습니다. 필수 frontmatter 필드는 `name`과 `description`이며, 특히 `description`을 구체적으로 작성하면(어떤 상황에서 쓰는 스킬인지, 트리거가 될 표현을 포함) 에이전트가 적절한 상황에서 스킬을 더 정확하게 찾아 사용합니다.

07-6. 백그라운드 큐레이터(Curator)

> ⚠️ ***부분 검증**: 이 페이지는 v0.16.0 소스 코드(`agent/curator.py` , `hermes_cli/curator.py` , `tools/skill_manager_tool.py` , `tools/skill_usage.py`)를 대조해 확인했습니다. 큐레이터는 7일 주기·비활성 가드로 백그라운드에서 조용히 실행되므로, 실제 자동 실행을 관찰하는 검증은 하지 않았습니다.*

- 한 줄 요약
- 작동 방식
- 보고서
- 기본 주기와 안전장치
- CLI 명령
- 모델 선택 — auxiliary.curator
- 실패와 복구
- 자기 진화 시스템과의 위치

스킬 라이브러리는 시간이 지날수록 늘어나기 마련입니다. 자동 생성된 스킬, 한 번 쓰고 잊은 스킬, 비슷한 작업을 다른 이름으로 흩어 둔 스킬이 쌓이면 어느 순간 어떤 스킬이 살아 있고 어떤 스킬이 죽어 있는지 알기 어려워집니다.

Curator(큐레이터)는 이 문제를 자동으로 정리하는 백그라운드 에이전트입니다. v0.12에서 게이트웨이의 정식 기능으로 들어왔습니다.

한 줄 요약

> *CLI 시작 시점과 게이트웨이 크론 티커에서 비활성·주기(기본 7일) 조건을 만족하면 깨어나, 스킬을 등급 매기고·합치고·치우는 백그라운드 리뷰 패스. 매 실행마다 사람이 읽는 보고서를 남기고, 잘못된 정리는 한 명령으로 롤백한다.*

작동 방식

큐레이터는 두 단계로 일합니다.

1단계 — 휴리스틱(LLM 호출 없음)

마지막 사용 시각 기준으로 스킬을 자동 전이시킵니다.

상태	의미
`active`	최근에 쓰인 스킬
`stale`	`stale_after_days` 동안 안 쓰인 스킬 (기본 30일)
`archived`	`archive_after_days` 동안 안 쓰인 스킬 (기본 90일). `~/.hermes/skills/.archive/` 로 이동

`pin` 된 스킬은 이 자동 전이를 받지 않습니다.

2단계 — LLM 리뷰 패스

활성 스킬 묶음을 보조 모델(`auxiliary.curator` 슬롯)에 보내, 다음을 판단합니다.

- 비슷한 스킬을 **합칠지(consolidated)** — 그러면 한쪽을 다른 쪽에 흡수시키고 원본은 아카이브
- 살아 있어도 가치가 낮은 스킬을 **솎아낼지(pruned)** — 아카이브로 보냄
- 살릴 만한 활성 스킬에는 사용 흔적을 반영해 가벼운 정리

리뷰 패스는「자동 생성」절의 세션 단위 백그라운드 리뷰와 마찬가지로 포크된 보조 에이전트가 수행합니다. 다만 세션 단위 리뷰가 `memory`·`skills` 도구로 제한되는 것과 달리, 큐레이터 포크는 스킬 디렉터리를 `.archive/` 로 옮기거나 참조 파일을 재배치하기 위해 스킬 도구에 더해 `terminal` 을 사용합니다.

보고서

매 실행마다 다음 두 파일이 생성됩니다.

```
~/.hermes/logs/curator/<YYYYMMDD-HHMMSS>/
├── run.json     # 기계 판독용: 전이 내역, 합쳐짐/솎임 분류, 점수
└── REPORT.md    # 사람이 읽는 한 페이지 요약
```

`REPORT.md` 마지막에는 "이번 실행에 무엇이 바뀌었나"가 한눈에 보이도록 정리됩니다. 의심스러운 전이가 있으면 이 파일에서 확인하고, 필요하면 아래 롤백으로 되돌립니다.

기본 주기와 안전장치

- **기본 주기**: **7일 (168시간)**. `curator.interval_hours` 설정으로 변경 가능
- **첫 실행 시점**: 처음 설치한 직후에는 라이브러리가 충분히 익기 전이므로, 설치 시점부터 한 주기(기본 7일)를 그대로 기다린 다음에야 첫 실제 패스가 돕니다. 그동안 무엇이 정리될지 미리 보고 싶다면 `hermes curator run --dry-`

`run`을 직접 호출합니다(상태·아카이브에 영향 없음)

- **idle 가드**: 최근까지 도구가 활발히 호출됐다면 큐레이터를 미룹니다(`curator.min_idle_hours`, 기본 2시간)
- **자동 백업**: 모든 실제 실행 직전에 `~/.hermes/skills/`의 tar.gz 스냅숏이 자동으로 만들어집니다
- **허브 스킬 보호**: 허브에서 받은 스킬은 큐레이터의 변경 대상에서 항상 제외됩니다. 번들 기본 스킬은 `curator.prune_builtins`(기본값 켜짐)에 따라 큐레이션 후보에 포함되며, 비활성 기간이 지나면 아카이브될 수 있습니다(삭제는 없음). `hermes update`로 다시 복사되지 않도록 별도 억제 목록으로 관리됩니다
- **핀 보호**: `pin`된 스킬은 자동 전이와 `skill_manage`의 **삭제**에서 제외됩니다. 패치·편집은 핀 여부와 무관하게 허용됩니다

CLI 명령

```
hermes curator status                # 현재 상태와 스킬 통계
hermes curator run                   # 지금 한 번 돌리기 (수동 실행은 기본적으로 LLM 패스가 끝날 때까지 대기)
hermes curator run --background      # LLM 패스를 백그라운드로 돌리고 즉시 반환
hermes curator run --dry-run         # 미리 보기 — 상태 변경/아카이브/통합 없이 보고서만
hermes curator pause                 # 큐레이터 일시 정지
hermes curator resume                # 재개
hermes curator pin <skill>           # 자동 전이로부터 보호
hermes curator unpin <skill>
hermes curator restore <skill>       # 아카이브된 스킬 되살리기
hermes curator backup                # 수동 스냅숏 생성
hermes curator backup --reason "before-cleanup"
hermes curator rollback --list       # 스냅숏 목록
hermes curator rollback              # 최신 스냅숏으로 복원
hermes curator rollback --id <id> -y # 특정 스냅숏으로 복원, 확인 생략
hermes curator archive <skill>       # 한 스킬을 지금 아카이브 (핀이면 거부)
hermes curator prune --days 90       # N일 이상 비활성 자동 생성 스킬 일괄 아카이브
hermes curator list-archived         # 아카이브된 스킬 목록
```

`hermes curator status`는 자동 생성된 스킬 목록을 사용 빈도(가장 많이 쓴 / 가장 안 쓴)로 정렬해서 보여줍니다. 라이브러리에 어떤 스킬이 살아 있고 어떤 스킬이 죽어 있는지 한눈에 파악할 수 있습니다.

모델 선택 — `auxiliary.curator`

큐레이터는 보조 모델 슬롯 중 `curator`를 사용합니다(2장 「클라우드 API 연동」과 「설정 파일」 참고). 라이브러리가 클수록 한 번의 패스가 토큰을 많이 씁니다(수백 개 후보 스킬에 대한 우산형 평가). 메인 모델로 그대로 돌리면 비용이 부담스러우므로, 보통은 저렴한 보조 모델을 지정합니다.

```
auxiliary:
  curator:
    provider: openrouter
    model: google/gemini-3-flash-preview
    timeout: 600   # 보조 모델 슬롯 중 가장 길게 잡혀 있음
```

`hermes model` → 보조 모델 → Curator 메뉴, 또는 웹 대시보드의 Models 탭에서도 직접 바꿀 수 있습니다.

실패와 복구

큐레이터가 잘못 판단해 멀쩡한 스킬을 솎아 버렸다면:

1. `hermes curator status`로 현재 상태 확인
2. 단일 스킬만 되살릴 거면 `hermes curator restore <name>`
3. 패스 전체를 되돌릴 거면 `hermes curator rollback --list`로 스냅숏을 본 다음, `hermes curator rollback --id <id>`

수동으로 잘 다듬어 둔 스킬은 미리 `hermes curator pin <name>`으로 고정해 두면 안전합니다.

자기 진화 시스템과의 위치

큐레이터는 12장 「자기 진화 시스템」에서 다루는 자기 진화 루프(메모리 + 스킬 자동 갱신)의 **장기 정리 단계**에 해당합니다.

```
턴 단위:    메인 대화에서 도구 사용
세션 단위:  백그라운드 리뷰 포크가 메모리/스킬 갱신 (7장 「자동 생성」)
주 단위:    큐레이터가 라이브러리 전체 재정렬 (이 절)
```

세 층 모두 같은 리뷰 포크 메커니즘 위에 있고, 시간 단위만 다릅니다. 자주 쓰는 스킬은 갈고닦이고, 오래 안 쓰는 스킬은 자연스럽게 정리되는 흐름이 만들어집니다.

> **처음에는 dry-run으로 감을 잡으세요**
>
> 설치 후 한 주기(7일) 동안 사용한 다음, 첫 자동 패스가 돌기 전에 `hermes curator run --dry-run`을 한 번 직접 호출해 `REPORT.md`를 읽어 보세요. 큐레이터가 무엇을 정리하려고 하는지 미리 파악한 뒤에 자동 실행을 받으면, 의도와 다른 결정을 줄일 수 있습니다.